CADT IDT

Lecture01

Introduction to Automata Theory

Automata Theory

Prepared by: Mr. POV Phannet

ABOUT ME

Lecturer: Mr. Pov Phannet

Position: Researcher-Lecturer, IDRI, CADT

Education:

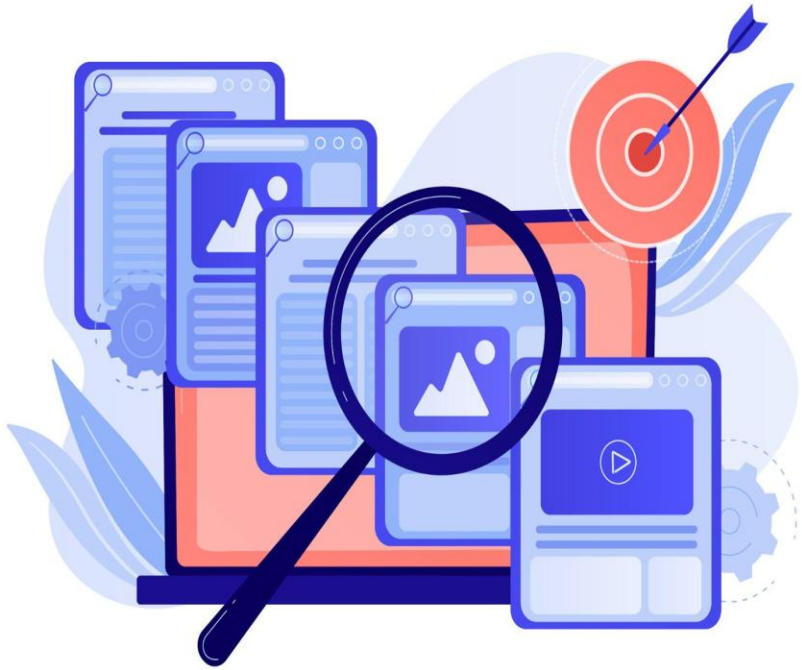
- 2018-2023: Bachelor's Degree in Computer Science, Institute of Technology of Cambodia
- 2023-2024: Master's Degree in Data Science, Institute of Technology of Cambodia

Email: phannet.pov@cadt.edu.kh

Telegram: pov_phannet

COURSE SCHEDULE

- Week 1: Introduction to Automata Theory
- Week 2: Languages and Operations on Languages
- Week 3: Deterministic Finite Automata
- Week 4: Non-deterministic Finite Automata
- Week 5: Deterministic and Minimization of Finite Automata
- Week 6: Regular Expression
- Week 7: Finite Automata and Regular Expression
- Week 8: Context-Free Grammars and Languages
- Week 9: Pushdown Automata
- Week 10: Turning Machines

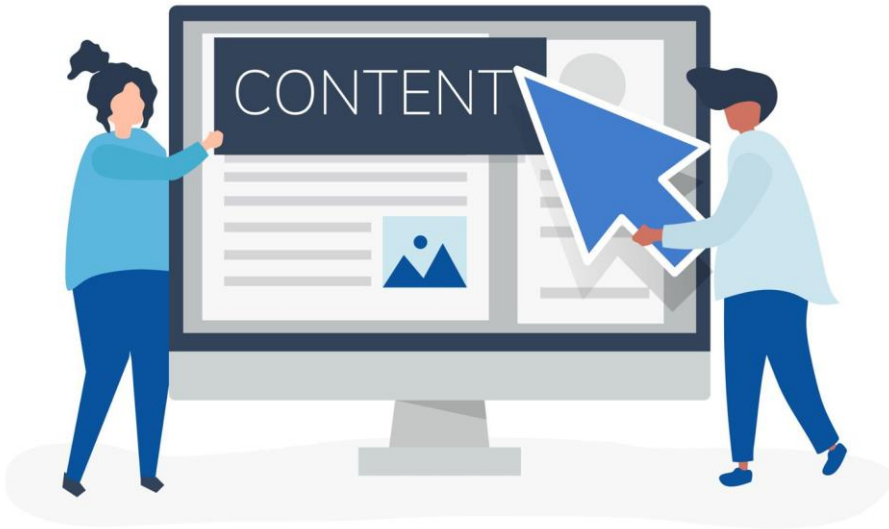


COURSE EVALUATION

Type of Evaluation	Ratio
Attendance and class participation	10%
Quizzes	10%
Lab practice assessments and Assignments	20%
Mid-Term Exam	20%
Final Exam	20%
Final Project	20%

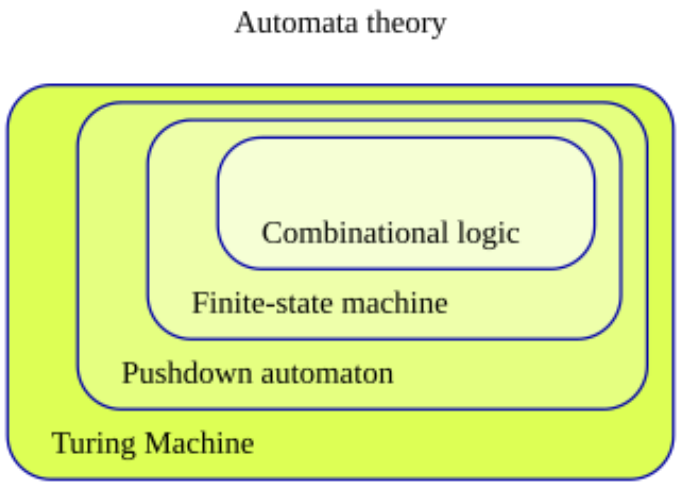
CONTENTS

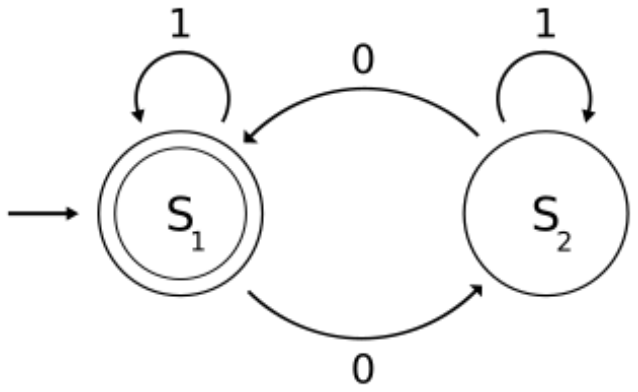
1. What is Automata Theory
2. Why study Automata Theory
3. Definitions, Theorems, and Proofs
4. Central Concepts of Automata Theory



1. WHAT IS AUTOMATA THEORY?

Automata Theory is a branch of the Theory of Computation. It deals with the study of abstract machines and their capacities for computation. An abstract machine is called the automata. It includes the design and analysis of automata, which are mathematical models that can perform computations on strings of symbols according to a set of rules.





2. WHY STUDY AUTOMATA THEORY

Automata theory deals with the definitions and properties of mathematical models of computation. It plays a role in several applied areas of computer science:

- Text processing
- Compilers (programming language)
- Artificial intelligence
- Hardware design

2. WHY STUDY AUTOMATA THEORY

2.1 Introduction to Finite Automata

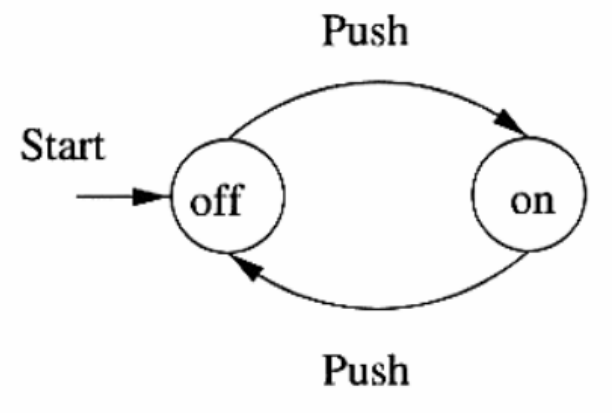
- System can be viewed as being of a finite number of state
- A state is used to remember a portion of system's history

2. WHY STUDY AUTOMATA THEORY

2.1 Introduction to Finite Automata

Example 1: on/off switch

- There are two states: “on” state and “off” state
- A button can be pressed to change state
- The states are represented by circles (named on and off)
- Arcs between states represent external influence on the system
- The “start state” designates the state in which the system is placed initially

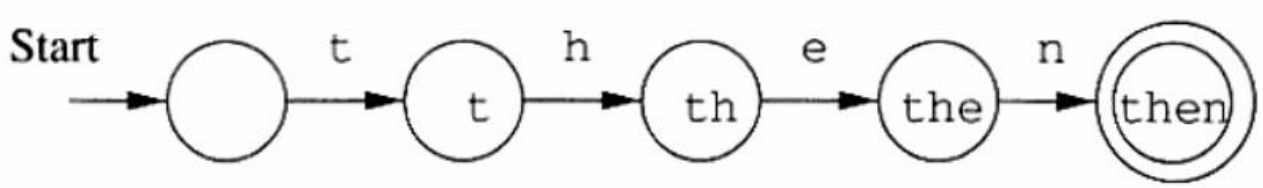


2. WHY STUDY AUTOMATA THEORY

2.1 Introduction to Finite Automata (cont.)

Example 2: lexical analyzer

- The system recognizes the keyword “then”
- There are five states
  - The start state corresponds to the empty string
  - The final state corresponds to the state named “then” which is entered when the input has spelled the word “then”



2. WHY STUDY AUTOMATA THEORY

2.2 Structural Representations

- Two important notions plays an important role in the study of automata and their application: regular expression and grammar
- **Regular Expression:** denote the structure of data, especially text string
  - Example: the UNIX-style regular expression [A-Z][a-z]*[ ][A-Z][A-Z] represents capitalized words followed by a space and two capital letters
- **Grammars:** are useful models when designing software that processes data with a recursive structure
  - Example: a parser that deals with arithmetic expression ($E \Rightarrow E + E$)

3. DEFINITIONS, THEOREMS, AND PROOFS

- **Definitions** describe the objects and notions that we use
- A **proof** is a convincing logical argument that a statement is true
- A **theorem** is a mathematical statement proved true
- Occasionally we prove statements that are interesting only because they assist in the proof of another, more significant statement. Such statements are called "**lemma**"
- A theorem or its proof may allow us to conclude easily that other, related statements are true. These statements are called **corollaries** of the theorems

4. CENTRAL CONCEPT OF AUTOMATA THEORY

4.1 Sets, Sequences, and Tuples

- A set is a group of objects represented as a unit
- A set may contain any type of object
  - Numbers
  - Symbols
  - Other sets
- Some examples
  - A set of number: {7, 12, 23}
  - A set of names: {Sok, Daro, Heng, Dara}
- The objects in a set are called elements or members
- The symbol $\in$ and $\notin$ denote set membership and nonmember ship
  - Example: $7 \in \{7, 12, 23\}$ and $8 \notin \{7, 12, 23\}$

4. CENTRAL CONCEPT OF AUTOMATA THEORY

4.1 Sets, Sequences, and Tuples (cont.)

- A sequence of objects is a list of these objects in some order
- ✓ Example: (7, 21, 57) is a sequence (the order is important here)
- As with sets, sequence may be finite or infinite. Finite sequences often are called **tuples**.
- A sequence with *k* elements is a **k-tuple**
- ✓ Example: (7, 21, 57) is a 3-tuple
- A 2-tuple is also called a **pair**

3. DEFINITIONS, THEOREMS, AND PROOFS

3.2 Alphabets

- An **alphabet** is a finite set (not empty) of elements called **symbols**.
- Some examples of alphabets:
  - $X = \{0, 1\}$ alphabet of binary number
  - $X = \{a, b, c, \ldots, z\}$ alphabet of English language which contains 26 symbols
  - $X = \{$ក, ខ, គ, ..., អ, ...$\}$ alphabet of Khmer characters

4. CENTRAL CONCEPT OF AUTOMATA THEORY

4.3 Strings

- A string over an alphabet X is a finite sequence of symbols from that alphabet
  - Example: w = 1001101 is a string over alphabet $X = \{0, 1\}$
- An empty string, written ε (epsilon), is a sequence that contains no symbol.
- If w i is a string over alphabet X, the length of w, written $|w|$, is the number of symbols that it contains
  - Example: $|abc| = 3$, $|\varepsilon| = 0$
- The a-length of a string w, written $|w|_a$,is the number of occurrences of symbol a in w
  - Example: $|abc|_a = 1$,$|abbab|_b = 3$
- $|x| = \sum_{a \epsilon X} |x|_a$
  - Example: $w = abccbb$ a string over $X = \{a, b, c\}$, $|w| = |w|_a + |w|_b + |w|_c 6 = 2 + 3 + 1 =$

4. CENTRAL CONCEPT OF AUTOMATA THEORY

4.3 Strings (cont.)

- Suppose we have two strings w_1 and w_2 over . The concatenation between w_1 and w_2, written $w_1 w_2$ (or $w_1 \cdot w_2$) is a new string obtained from adding w_2 to w_1

➢ Example: $w_1 = ab, w_2 = cd \Rightarrow w_1 w_2 = adcd$

$$\varepsilon \cdot w = w \cdot \varepsilon = w$$

- Some remarks:

✓ $|w_1 w_2| = |w_1| + |w_2|$

✓ $|\varepsilon w| = |w\varepsilon| + |w|$

- An n-power of a string w, written w^n, is defined by:

$$\begin{cases} w^0 = \varepsilon \\ w^n = w \cdot w^{n-1}, \quad \forall n > 0 \end{cases}$$

➢ Example: $w = ab, w^2 = abab, w^3 = ababab, \ldots$

4. CENTRAL CONCEPT OF AUTOMATA THEORY

4.3 Alphabets

- A mirror image of a string $w = a_1 a_2 a_3 \ldots a_n$ is a new string written w^R such that

$$w^R = a_n a_{n-1} \ldots a_2 a_1$$

- Example: $w = abc \Rightarrow w^R = cba$

- The lexicographic ordering of string is the same as the dictionary ordering, except that shorter strings precede longer strings

- Example: the lexicographic order of all strings over alphabet $X = \{a, b\}$ is

$(\epsilon, a, b, aa, ab, bb, aaa, \ldots)$

4. CENTRAL CONCEPT OF AUTOMATA THEORY

4.3 Alphabets

Two strings w_1 and w_2 are equal if they are of the same length and contain the same sequence of symbols in the same order

- A string w_1 is a prefix of w_2 if it exists a non empty string w_3 such that $w_1 w_3 = w_2$
- A string w_1 is a suffix of w_2 if it exists a non empty string w_3 such that $w_3 w_1 = w_2$
- A string w_1 is a substring of w_2 if it exists w_3 and w_4 such that $w_3 w_1 w_4 = w_2$

REFERENCES

- https://www.geeksforgeeks.org/theory-of-computation-automata-tutorials/
- Heng, S. (2007). Théories des automates. Institute of Technology of Cambodia, Department of Computer Science.
- Hopcroft, J., Motwani, R., & Ullman, D. (2006). Introduction to Automata Theory, Languages, and Computation. Addison Wesley.
- Sipser, M. (2006). Introduction to the Theory of Computation. Thomas Course Technology.